

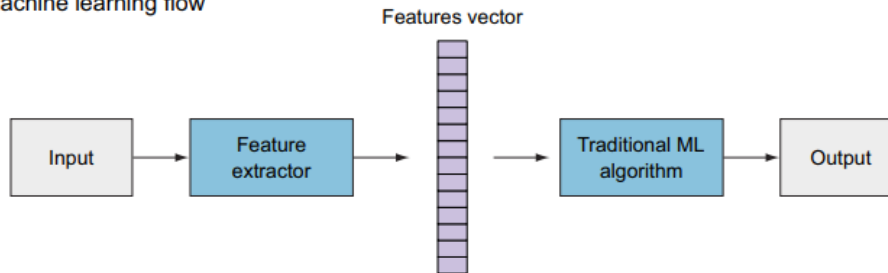
Artificial Neural Networks

Chapter 10: Introduction to Artificial Neural Networks with Keras, Chapter 11: Training Deep Neural Networks
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Shallow Learning vs Deep Learning

- Traditional ML algorithms require manual feature extraction.
- While specifying a linear regression or classification model, the designer chooses the features and parameters on which the model is trained – **handcrafted features**.
- This is **conventional machine learning or shallow learning**.
- Becomes **increasingly complex** with large number of features.

Traditional machine learning flow



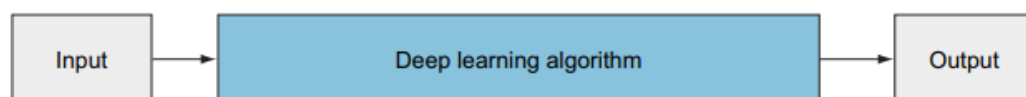
- A deep neural network automatically extracts features by passing the input image through its layers.
- It extracts and learns features from data by itself regardless of its linear or non-linear nature – **automatic features**.
- This ability makes it **powerful and magical**.
- This is termed as deep learning, here algorithms/layers are stacked in a hierarchy of increasing complexity and abstraction.
- It is particularly useful in:
 - solving complex non-linear classification problems.
 - processing and classification of images and text.

If you plot your data and a straight line can't separate the classes, the problem is non-linear.

Linear → one straight line works

Non-linear → you need curves, circles, or complex shapes

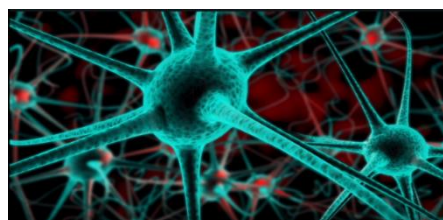
Deep learning flow



Artificial Neural Networks

- Artificial neural network (ANNs) are at the very core of Deep Learning.
- They are versatile, powerful, and scalable, making them ideal to tackle large and highly complex Machine Learning tasks, such as classifying billions of images (e.g., Google Images), powering speech recognition services (e.g., Apple's Siri), recommending the best videos to watch to hundreds of millions of users every day (e.g., YouTube), or learning to beat the world champion at the game of Go by playing millions of games against itself (DeepMind's Alpha- Zero).
- ANNs involves algorithms that try to mimic the human brain.
- This technique was widely used in 80s and early 90s, however its popularity diminished in late 90s.
- It has **recent resurgence** as the **state-of-the-art technique** for many applications.

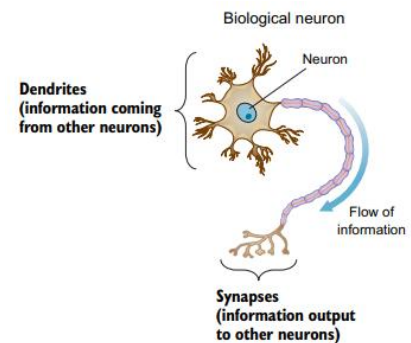
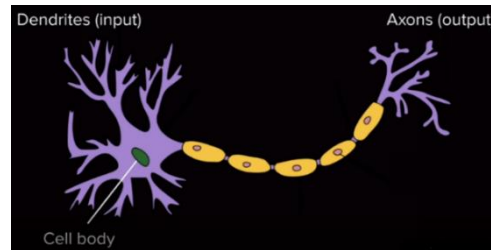
Inspiration – The Human Brain



- Human brain comprises of approximately 10 billion interconnected neurons.
- Each neuron is connected to about 10000 other neurons.
- Neurons pass electrical and/or chemical signals to one another.

The Biological Neuron

- A dendrite is an input unit which receives signals from other neurons.
- An axon is the output unit which passes signals to other neurons.
- If the sum of signals received from through dendrites of neuron from upstream neurons is strong enough, then these signals are passed to the downstream neurons connected through axons; the neuron is said to be activated.
- The key point is that the neuron only fires if the signals received at cell body exceeds a certain threshold.
- Individually a neuron is a very simple processing unit – it either fires or not fires.
- In the nutshell, a biological neuron receives electrical signals from its dendrites, modulates the electrical signals in various amounts, and then fires an output signal through its synapses only when the total strength of the input signals exceeds a certain threshold. The output is then fed to another neuron, and so forth.
- Synapse is the connection point between two neurons.



The Perceptron – The Artificial Neuron

- To model the biological neuron phenomenon, the artificial neuron performs two consecutive functions:
 - It calculates the weighted sum of the inputs to represent the total strength of the input signals, and
 - It applies an activation function to the result to determine whether to fire the output 1 or not.
- Activation function is the decision making unit of a perceptron.
- Not all input features are equally useful or important, therefore each input node is assigned a weight value, called its connection weight, to reflect its importance.

Input vector

- The feature vector that is fed to the neuron.
- It is usually denoted with an uppercase X to represent a vector of inputs ($x_1, x_2, \dots, x_n$).

Weights vector

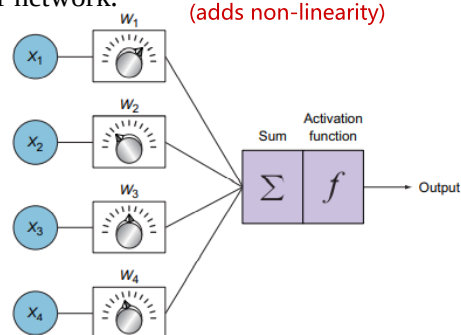
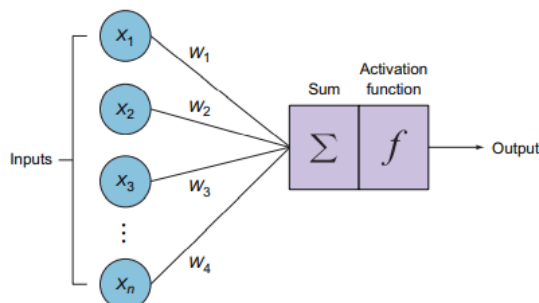
- Each input x_i is assigned a weight value w_i that represents its importance to distinguish different input data points.

Neuron functions

- The calculations performed within the neuron to modulate the input signals - the weighted sum and activation function.
- There are different activation functions that can be used.
- For a step function, the output is either 0 or 1; other activation functions produce probability output or float numbers.

Output

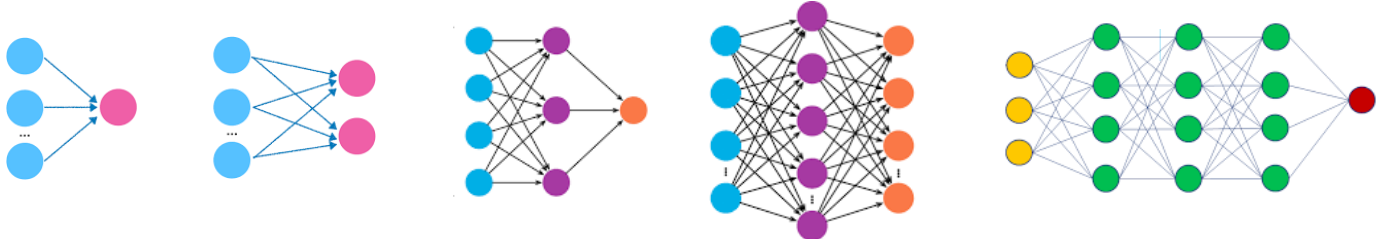
- The output node represents the perceptron prediction.
- Controlled by the type of activation function you choose for your network.



Neuron function → computes a weighted sum of inputs
Activation function → decides the final output (adds non-linearity)

Layers in Artificial Neural Network

- Neural networks are composed of nodes (perceptrons) which are connected to each other.
- The strengths of their connections are called weights.
- Learning is the adjustment of these weights.
- A very common neural network architecture is to stack the neurons in layers on top of each other, called layers.
- Each layer can have different number of neurons.
- Layers are connected to each other by weight connections.
- The more complex (more layers) our network is, the better it learns the features of our data.
- Some arrangements of neurons in a neural network:



Fully Connected Network

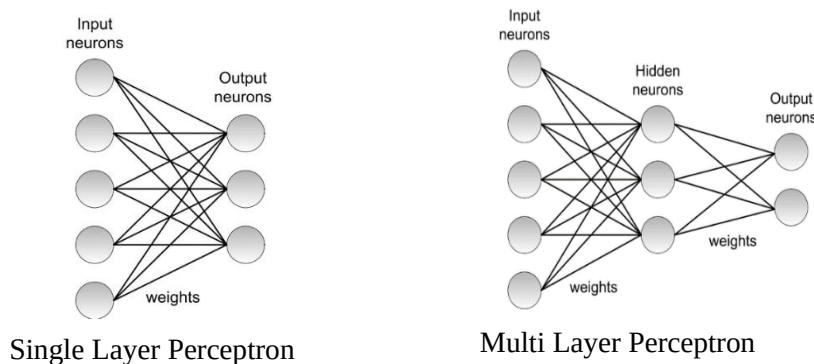
- If each node in a layer is connected to all nodes in the previous layer, then it is called a fully connected network.

Single Layer Perceptron

- The architecture consists of a layer on input neurons fully connected to a single layer of output neurons.

Multilayer Perceptron

- The architecture consists of at least one hidden layer in between the input and output layers.
- A neural network with two or more hidden layers is called a deep neural network.
- The more the layers, the deeper the network!



Multilayer Perceptron (MLP) Architecture

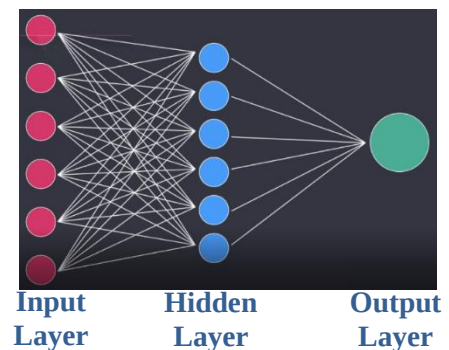
- Three types of layers in a MLP are input layer, hidden layer, and output layer.

Input layer

- The input layer takes raw input from the domain.
- It contains the feature vector; each node represents a feature.
- No computation is performed at this layer.
- Nodes here just pass on the information (features) to the hidden layer.

Hidden layers

- The hidden layer provides an abstraction to the neural network.
- It performs all kinds of computation on the features entered through the input layer and transfers the result to the output layer.
 - A hidden layer is where the core of the feature learning process takes place.
- They are called hidden layers because we don't see or control the input going into these layers or the output.
 - All we do is feed the feature vector to the input layer and see the output coming out of the output layer.
- A network can have one or more hidden layers (technically, as many as you want).
- Each layer has one or more neurons (again, as many as you want).



- A neural network with two or more hidden layers is called a deep neural network.
- The more the layers, the deeper the network!
- Your main job, as a machine learning engineer, is to design these layers.

Weight connections (edges)

- Weights are assigned to each connection between the nodes to reflect the importance of their influence on the final output prediction.

Output layer

- Output Layer is the final layer of the network that brings the information learned through the hidden layer and delivers the final value as a result.
- Depending on the setup of the neural network, the final output may be a real-valued output (regression problem) or a set of probabilities (classification problem).
- This is determined by the type of activation function we use in the neurons in the output layer.
- If there are more than one outputs in a model, then we can have more than one output neurons.

Feedforward Propagation to calculate the result

- The flow of information occurs in the forward direction.
- Each neuron takes input and multiply by the neuron's weight, add bias, feed the result to the activation function, take the output and pass it to the next layer neurons.
- The output produced by the final layer is compared with the actual output to calculate error through some cost function.

Backpropagation to minimize the error

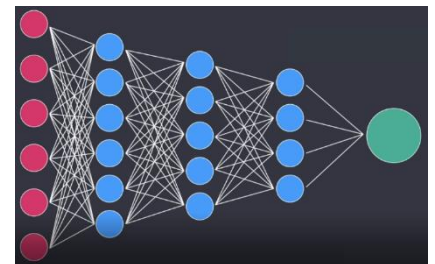
- The weights of the network connections are repeatedly adjusted to minimize the difference between the actual output vector of the net and the desired output vector.
- It aims to minimize the cost function by adjusting the network's weights and biases.
- The cost function gradients determine the level of adjustment with respect to the parameters.
- In a neural network the weights are adjusted in backward direction, hence the term backpropagation.

Bias

- Adjusting weights, changes the model function.
- The bias value allows the activation function to be shifted to the left or right, to better fit the data.
- It is a constant added to the activation function.
- A bias node can be added to input layer and all hidden layers; output layer does not have a bias node.
- Like input nodes, no calculation takes place in bias nodes.
- A bias node does not take any input from preceding layer.

An Intuitive Understanding of Working of the Hidden Layer

- The early layers detect simple patterns to learn low-level features (straight lines).
- Later layers detect patterns within patterns to learn more complex features and shapes, then patterns within patterns within patterns, and so on.
- Thus the hidden layers are stacked to learn complex features from each other until the model fits the data.
- So when you are designing your neural network, if your network is not fitting the data, one solution could be adding more hidden layers.
- The general rule is this: the deeper your network is, the more it will fit the training data. solution of underfitting
- But too much depth is not a good thing, because it may result in overfitting; also, it becomes more computationally expensive. drawback of overfitting



Hyperparameters in a Neural Network

Hidden Layers and Number of Nodes in each Hidden Layer

- The two main hyper-parameters that control the architecture or topology of a neural network are the number of layers, and the number of nodes in each hidden layer.
- The most reliable way to configure these hyper-parameters for a specific modeling problem is via systematic experimentation with a robust test harness.
- Start with a small network, and observe the network performance; then go on adding layers until you get satisfying results.
- However there are certain rules of thumb that can provide a good starting point, a few are given in the following.

Choosing Hidden Layers

- If the data is linearly separable then no hidden layers are needed at all.
- If data is less complex and is having fewer dimensions or features then neural networks with 1 to 2 hidden layers would work.
- If data is having large dimensions or features then to get an optimum solution, 3 to 5 hidden layers can be used.
- Too many hidden layers may sometimes lead to overfitting.

Choosing Nodes in Hidden Layers

- The number of hidden neurons should be between the size of the input layer and the output layer.
- The number of hidden neurons should keep on decreasing in subsequent layers to get more and more close to pattern and feature extraction and to identify the target class.

Activation Function

- Decides how the inputs are transferred to the output.
- All hidden layers usually use the same activation function.
- However, the output layer will typically use a different activation function from the hidden layers.
- The choice depends on the goal or type of prediction made by the model.

Error Function

- Measures how far the network's prediction is from the true label.
- Mean square error is common for regression problems, and cross-entropy is common for classification problems.

Optimizer

- Optimization algorithms are used to find the optimum weight values that minimize the error.
- There are several optimizer types to choose from, for example batch gradient descent, stochastic gradient descent, and mini-batch gradient descent, Adam and RMSprop.

Learning rate

- One of the optimizer's input parameters that is tuned.
- Theoretically, a learning rate that is too small is guaranteed to reach the minimum error (if you train for infinity time).
- A learning rate that is too big speeds up the learning but is not guaranteed to find the minimum error.
- The default learning rate value of the optimizer in most deep learning libraries is a reasonable start to get decent results.
 - From there, go down or up by one order of magnitude until optimum results are achieved.

Epoch

- An epoch is when an entire dataset passes through the network a single time.
- Number of epochs is the number of times the entire training dataset is shown to the network while training.
- Generally the more the epochs, the better the model fit is; after a certain epochs, the model starts overfitting.

Batch

- The batch size is a hyperparameter that defines the number of samples to work through before updating the internal model parameters.
- Bigger batch sizes learn faster but require more memory space.
- A good default for batch size might be 32. Also try 64, 128, 256, and so on.

Complexity of Artificial Neural Networks

- A neural network can solve linear as well as non-linear problem, but should not be applied to every problem due to certain complexities and disadvantages.

Black Box Nature

- So the main idea is that the layers in a neural network break down data into hierarchy, but we actually do not know what goes on under the hood, which features are actually generated.
- It is a black box!
- We do not know the rationale behind the decision made. Any application that requires reasoning is completely beyond what current deep learning techniques can do, even with large data.

Huge Costs

- Deep learning incurs huge costs. The cost comes in two forms, the amount of data required, and the amount of computations required.
- More powerful and accurate models need more parameters. The more the parameters, the more the data required for training. parameters are the values the model learns from data to make predictions.
- One rule of thumb is that the data should be ten times more than the number of parameters to be trained.
- The number of parameters goes up massively with the size of the network.
- The more parameters are trained, the more computations are done, thus escalating hardware requirements.
- Memory requirements (both primary and secondary) are also huge.
- Multicore high-performing processing units are required to ensure improved efficiency and decreased time consumption.

Inflexibility

- Once trained, deep learning models become inflexible and cannot handle multitasking.
- They can deliver efficient and accurate solutions but only to one specific problem.
- Even solving a similar problem would require retraining the system.

High-end Processing Units

General Purpose Graphics Processing Unit (GPGPU)

- While GPUs were originally designed primarily for the purpose of rendering images, GPGPUs can now be programmed to direct that processing power toward addressing scientific computing needs as well.
- If a graphics card is compatible with any particular framework that provides access to general purpose computation, it is a GPGPU.
- Essentially all modern GPUs are GPGPUs.
- Two such popular parallel programming platforms are CUDA (Compute Unified Device Architecture) and OpenCL (Open Computing Language).
- CUDA is a parallel computing platform and application programming interface model created by NVIDIA.
 - It allows software developers and software engineers to use a CUDA-enabled graphics processing unit (GPU) for general-purpose processing.
- These CUDA cores are highly beneficial and evolutionary in the field of artificial intelligence.
- The compute capability (CC) of a GPU determines its general specifications and available features.
- Some popular products of NVIDIA are GeForce, TITAN, Quadro, and RTX.

Web-based GPGPUs

- Multicore high-performing graphics processing units (GPUs) and other similar processing units are required to ensure improved efficiency and decreased time consumption.
- However, these units are expensive and use large amounts of energy.
- Some low cost web-based GPU services available include Amazon Web Service (AWS), Microsoft Azure and Google CoLab.

Activation Functions

- Activation functions are also referred to as transfer functions.
- An activation function is placed at the end of each perceptron to decide whether to activate this neuron.
- It decides whether the neuron's input to the network is important or not in the process of prediction using simpler mathematical operations.
- The role of the Activation Function is to derive output from a set of input values fed to a node (or a layer).
- Activation function helps the neural network to use important information while suppressing irrelevant data points.
- Another purpose of the activation function is to introduce nonlinearity into the network.

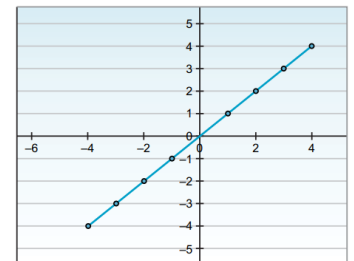
non linearity means after activation function Output is no longer a straight-line transformation
The network can learn curves, complex shapes, patterns

Some commonly used activation functions are as follows:

- Linear transfer function
- Binary step function
- Sigmoid function
- Softmax function
- Hyperbolic tangent function
- Rectifier linear function (ReLU)
- Leaky ReLU

Linear Transfer Function

- A linear transfer function indicates that the function passes a signal through unchanged.
- Also called an identity function.
- Mathematically, $f(z) = z$
- Can be used in output layer for regression tasks; however rarely used.

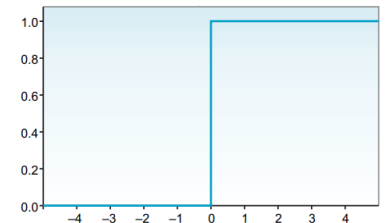


Binary Step Function

- Produces a binary output, 0 or 1.
- The input fed to the activation function is compared to 0.
- If the input is greater than 0, then the output is 1, i.e. the neuron is activated.
- Else the output is 0, the neuron is deactivated and its output is not passed on to the next layer.

- Mathematically, $f(z) = \begin{cases} 0, & \text{for } z < 0 \\ 1, & \text{for } z \geq 0 \end{cases}$

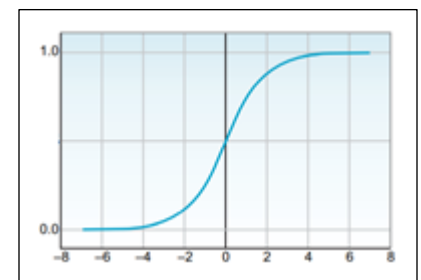
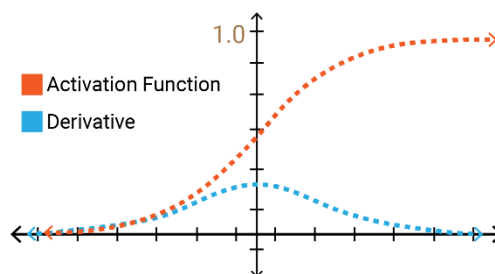
- Used for binary classification tasks.



Sigmoid Function

- This function takes any real value as input and outputs values in the range of 0 to 1.
- Mathematically, $f(z) = \frac{1}{1+e^{-z}}$
- It is often used in output layer of binary classifiers to predict the probability of a class when there are two classes.
- Not preferred for hidden layers as it causes vanishing gradient problem. notes : pile 11
- Derivative of sigmoid function is $f'(z) = f(z)(1 - f(z))$.

$$\begin{aligned} f(z) &= 0, f'(z) = 0 \\ f(z) &= 1, f'(z) = 0 \\ f(z) &= 0.5, f'(z) = 0.25 \end{aligned}$$

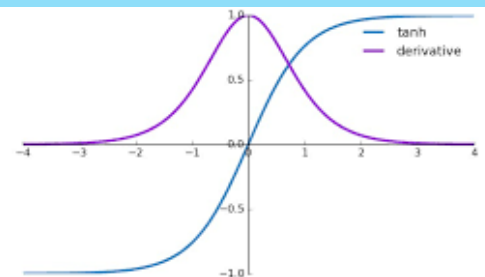
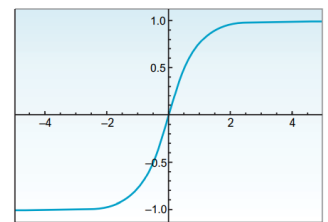


Softmax Function

- The softmax function is a **generalization of the sigmoid function**; described as a combination of multiple sigmoids.
- It is used to obtain **classification probabilities** when there are more than two classes; works fine for two classes as well.
- It calculates the relative probabilities, **returns the probability of each class**.
- It forces the outputs of a neural network to sum to 1 (for example, $0 < \text{output} < 1$).
- Mathematically, $(z_j) = \frac{e^{z_j}}{\sum_i e^{z_i}}$, $i, j = 1, 2, \dots, p$, where p is the number of classes.
- It is most commonly used as an **activation function for the output layer** of the neural network in the case of **multi-class classification**.
- Note that softmax activation function is used for **multiclass outputs**, where each instance can belong only to a single class in a multiclass classification. For multilabel outputs, where the outputs are independent of each other, it does not make sense that the probabilities add up to 1. For example, if the two possible outputs in an email spam filter are spam/non-spam and urgent/not-urgent. These two outputs are not mutually exclusive. An email can be non-spam and urgent at the same time.

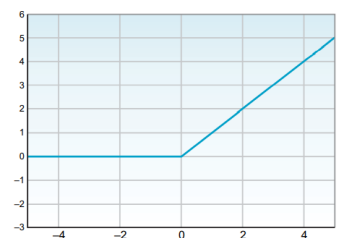
Hyperbolic Tangent (Tanh – *tangent hyperbolicus*) Function

- The hyperbolic tangent function is a **shifted version of the sigmoid version**.
- Instead of squeezing the signal values between 0 and 1, tanh squishes all values to the range **-1 to 1**.
- Mathematically, $f(z) = \tanh(z) = \frac{\sinh(z)}{\cosh(z)} = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
- Not preferred for hidden layers as it causes vanishing gradient problem.
- However if used in hidden layers, tanh almost always works better than the sigmoid function in hidden layers because the gradient of tanh is four times greater than t' problem of vanishing gradient a bit less intense.
- Derivative of hyperbolic tangent function is $f'(z) = (1 - f(z)^2)$.
- $f(z) = -1, f'(z) = 0$
- $f(z) = 1, f'(z) = 0$
- $f(z) = 0, f'(z) = 1$



Rectified Linear Unit (ReLU) Function

- The rectified linear unit (ReLU) activation function **activates a node only if the input is above zero**.
- If the input is below zero, the output is always zero; but when the input is higher than zero, it has a **linear relationship with the output variable**.
- Mathematically, $f(z) = \max(0, z) = \begin{cases} 0, & \text{for } z \leq 0 \\ z, & \text{for } z > 0 \end{cases}$
- It is the most commonly used function for hidden layers because of several reasons.
 - It **takes less time to learn** and is **computationally less expensive** than other common activation functions like tanh, sigmoid, because it outputs 0 whenever its input is negative, **fewer neurons will be activated, leading to network sparsity and thus higher computational efficiency**.
 - It involves **simpler mathematical operations** compared to tanh and sigmoid, thereby boosting its computational performance further.
 - It **does not cause the vanishing gradient problem** like sigmoid and tanh functions.
- While the characteristic of network sparsity gives ReLU its strengths, it becomes a problem when most of the inputs to these ReLU neurons are in the negative range, causing the **dying ReLU problem**.

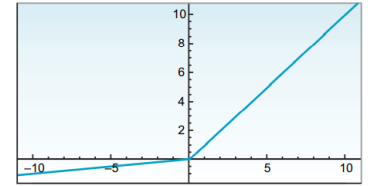


- Derivative of ReLU function is $f'(z) = \begin{cases} 0, & \text{for } z < 0 \\ 1, & \text{for } z > 0 \\ \text{Undefined (or set to 1 or 0 in practice),} & \text{for } z = 0 \end{cases}$

Network sparsity refers to a property of a neural network where most of the connections (weights) between neurons are zero or inactive, meaning only a small fraction of the neurons and connections are actively contributing at a time.

Leaky ReLU Function Dying relu solution

- Leaky ReLU is a ReLU variation that tries to mitigate the dying ReLU issue.
- Instead of having the function be zero when $x < 0$, leaky ReLU introduces a small negative slope (around 0.01) when (x) is negative.
- Mathematically, $f(z) = \max(0.01z, z) = \begin{cases} 0.01z, & \text{for } z < 0 \\ z, & \text{for } z \geq 0 \end{cases}$
- Different values can be tried in place of 0.01 treating it as another hyperparameter, for example, 0.1, 0.02, etc. But that would be overkill, since there already are other, bigger problems to worry about.
- It usually works better than the ReLU function, although it's not used as much in practice.



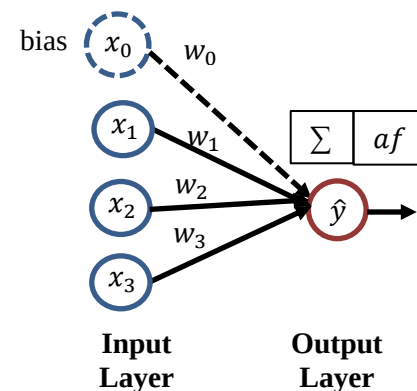
Training a Neural Network

- Training in a neural network consists of two main steps:
 - Feedforward Propagation in a Neural Network**
 - Information flows in the forward direction, from the input layer to the output layer.
 - Each neuron takes input values, multiplies them by corresponding weights, adds a bias term, passes the result through an activation function, and forwards the output to neurons in the next layer.
 - The final layer produces the network's predicted output, which is compared with the true output using a cost (loss) function to compute the error.
 - Backpropagation in a Neural Network**
 - The computed error is propagated backward through the network.
 - Gradients of the loss function with respect to each weight and bias are calculated.
 - These gradients are then used to update the weights and biases to minimize the loss.

Information Flow in Single Layer Perceptron (SLP)

Consider the given neural network.

- Number of features: 3 (x_1, x_2, x_3)
- Target (output), y : 1
- Number of data samples, n
- Predicted output, $\hat{y}$
- $x_0 = 1$, called the bias unit.
- Training parameters: 4 (w_0, w_1, w_2, w_3)
- Activation function, af
- Cost function (for n input samples), $J(w) = y^{(i)} - \hat{y}^{(i)}$
- Objective: minimize loss function, $J(w)_{min}$
- Optimizer: Stochastic Gradient descent ($w_{new} = w - \eta \frac{\partial J}{\partial w}$, where η is the learning rate)



Feedforward Propagation in SLP

- $\hat{y} = af(w_0x_0 + w_1x_1 + w_2x_2 + w_3x_3)$

Vector Form

- $w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \end{bmatrix}$ and $x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$
- $\hat{y} = w^T x$

Backpropagation in SLP

- Repeat until converge:

$$w_{0_new} = w_0 - \eta \frac{\partial J}{\partial w_0}$$

$$w_{1_new} = w_1 - \eta \frac{\partial J}{\partial w_1}$$

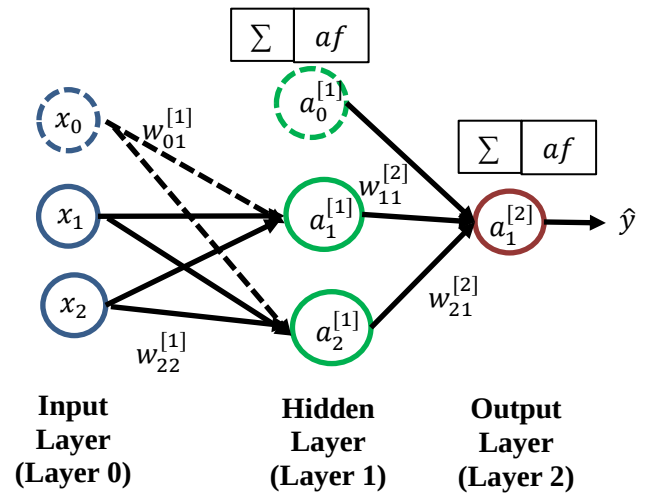
$$w_{2_new} = w_2 - \eta \frac{\partial J}{\partial w_2}$$

$$w_{3_new} = w_3 - \eta \frac{\partial J}{\partial w_3}$$

Information Flow in Multi Layer Perceptron (MLP)

Consider the given neural network.

- Number of features: 2 (x_1, x_2)
- Target (output), y : 1
- Number of data samples, n
- Predicted output, $\hat{y}$
- x_0 and $a_0^{[1]}$ are the bias units.
- Training parameters: 9
 $(w_{01}^{[1]}, w_{02}^{[1]}, w_{11}^{[1]}, w_{12}^{[1]}, w_{21}^{[1]}, w_{22}^{[1]}, w_{01}^{[2]}, w_{11}^{[2]}, w_{21}^{[2]})$
- Activation function, af
- Cost function (for i^{th} example),
 $J(w) = y^{(i)} - \hat{y}^{(i)}$
- Objective: minimize loss function, $J(w)_{min}$
- Optimizer: Stochastic Gradient descent ($w_{new} = w - \eta \frac{\partial J(w)}{\partial w}$),
 where η is the learning rate)
- Each layer has its own weight matrix.
- Dimension of weight matrix: $x \times y$, where x is the number of units in layer $j - 1$ (including the bias node) and y is the number of units in layer j (excluding the bias node)



Notation

$a_i^{[j]}$: Activation unit of node i in layer j .

$w_{xy}^{[j]}$: Weight between node x of layer $j - 1$ and node y of layer j .

Feedforward Propagation in MLP

Input Layer (Layer 0)

- $x_0 = 1$
- x_1 = feature 1 input.
- x_2 = feature 2 input.

Hidden Layer (Layer 1)

- $a_0^{[1]} = 1$
- $a_1^{[1]} = af(w_{01}^{[1]}x_0 + w_{11}^{[1]}x_1 + w_{21}^{[1]}x_2)$
- $a_2^{[1]} = af(w_{02}^{[1]}x_0 + w_{12}^{[1]}x_1 + w_{22}^{[1]}x_2)$

Output Layer (Layer 2)

- $\hat{y} = a_1^{[2]} = af(w_{01}^{[2]}a_0^{[1]} + w_{11}^{[2]}a_1^{[1]} + w_{21}^{[2]}a_2^{[1]})$

Vector Form

- $w^{[1]} = \begin{bmatrix} w_{01}^{[1]} & w_{02}^{[1]} \\ w_{11}^{[1]} & w_{12}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} \end{bmatrix}$, $x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$, and $a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \end{bmatrix}$
- $a^{[1]} = w^{[1]T}x$
- $w^{[2]} = \begin{bmatrix} w_{01}^{[2]} \\ w_{11}^{[2]} \\ w_{21}^{[2]} \end{bmatrix}$ and $a^{[1]} = \begin{bmatrix} a_0^{[1]} \\ a_1^{[1]} \\ a_2^{[1]} \end{bmatrix}$; $\hat{y} = w^{[2]T}a^{[1]}$

The Gradient Descent Algorithm for MLP

- $J(w) = y^{(i)} - \hat{y}^{(i)}$, $\hat{y} = a_1^{[2]} = af(w_{01}^{[2]}a_0^{[1]} + w_{11}^{[2]}a_1^{[1]} + w_{21}^{[2]}a_2^{[1]})$

- Repeat until converge: (or Repeat for the given number of epochs)

$$w_{01_new}^{[2]} = w_{01}^{[2]} - \eta \frac{\partial J(w)}{\partial w_{01}^{[2]}}$$

$$w_{11_new}^{[2]} = w_{11}^{[2]} - \eta \frac{\partial J(w)}{\partial w_{11}^{[2]}}$$

$$w_{21_new}^{[2]} = w_{21}^{[2]} - \eta \frac{\partial J(w)}{\partial w_{21}^{[2]}}$$

$$w_{01_new}^{[1]} = w_{01}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{01}^{[1]}}$$

$$w_{02_new}^{[1]} = w_{02}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{02}^{[1]}}$$

$$w_{11_new}^{[1]} = w_{11}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{11}^{[1]}}$$

$$w_{12_new}^{[1]} = w_{12}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{12}^{[1]}}$$

$$w_{21_new}^{[1]} = w_{21}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{21}^{[1]}}$$

$$w_{22_new}^{[1]} = w_{22}^{[1]} - \eta \frac{\partial J(w)}{\partial w_{22}^{[1]}}$$

Backpropagation in MLP

- Due to stacking of neurons in layers and use of multiple layers, finding derivative of cost function w.r.t. a certain weight is not as simple as it was in linear networks; it is done by using chain rule.
- These equations are worked out backwards using Back Propagation algorithm.
- Backpropagation algorithm is probably the most fundamental building block in a neural network.
- It was first introduced in 1960s and almost 30 years later (1989) popularized by Rumelhart, Hinton and Williams in a paper called *Learning representations by back-propagating errors*.
- The algorithm is used to effectively train a neural network through a method called chain rule.
- It aims to minimize the cost function by adjusting the network's weights and biases.
- The weights of the network connections are repeatedly adjusted to minimize the difference between the actual output vector of the net and the desired output vector.
- In a neural network the weights are adjusted in backward direction, hence the term backpropagation.

- Let $\delta_j^{[l]}$ represent the error of node j in layer l . $\delta^{[l]}$ is the vector containing error values for all nodes in layer l .
- For the output layer, this error is calculated as follows:
 $\delta^{[l]} = \hat{y} - y$
- For hidden layers, this error is calculated as follows:
 $\delta^{[l]} = w^{[l+1]} \delta^{[l+1]} * a'^{[l]}$, where $*$ represents one-to-one multiplication, called Hadamard product (element-wise product), and $a'^{[2]}$ is the derivative of $a^{[2]}$ w.r.t. to the summation function.
- These error values are now used to compute the derivatives of the cost function w.r.t the respective weights.

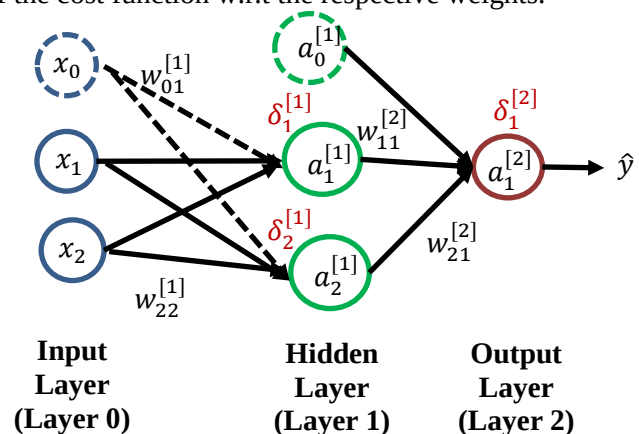
Layer 2:

- $\delta_1^{[2]} = \hat{y} - y$

Layer 1:

- $\delta_0^{[1]} = w_{01}^{[2]} \delta_1^{[2]} a_0'^{[1]}$ (not needed)
- $\delta_1^{[1]} = w_{11}^{[2]} \delta_1^{[2]} a_1'^{[1]}$
- $\delta_2^{[1]} = w_{21}^{[2]} \delta_1^{[2]} a_2'^{[1]}$

chain rule when 2 outputs?



Vector Form

- Hence, $\delta^{[2]} = \begin{bmatrix} \delta_1^{[2]} \end{bmatrix}$ and $\delta^{[1]} = \begin{bmatrix} \delta_1^{[1]} \\ \delta_2^{[1]} \end{bmatrix}$
- Hence, equivalent computationally efficient equations for the derivatives to devise Gradient Descent algorithm

$$\frac{\partial J(w)}{\partial w_{01}^{[2]}} = a_0^{[1]} \delta_1^{[2]}$$

$$\frac{\partial J(w)}{\partial w_{11}^{[2]}} = a_1^{[1]} \delta_1^{[2]}$$

$$\frac{\partial J(w)}{\partial w_{21}^{[2]}} = a_2^{[1]} \delta_1^{[2]}$$

$$\frac{\partial J(w)}{\partial w_{01}^{[1]}} = x_0 \delta_1^{[1]}$$

$$\frac{\partial J(w)}{\partial w_{02}^{[1]}} = x_0 \delta_2^{[1]} \dots\dots \text{and so on.}$$

The Finalized Gradient Descent Algorithm for MLP:

- For complete training set having n entries, the Gradient Descent algorithm is as follow:
 1. Initialize the weights and biases to some random values.
 2. For each training example
 - i. Compute output of all activation functions (feedforward computations).
 - ii. Compute error of the output layer.
 - iii. Compute error for hidden layers by backpropagating the errors.
 3. Find new weight values using cumulative errors from all training examples.
 4. Update all the weights.